ZENIKA · SPEED TRAINING IA · DÉVELOPPEURS · 4H

Développeur IA — Foundation

Du vibe coding au développement piloté par agent

L’agent ne devine pas : il prolonge ce qu’on lui donne. Le dev reste pilote.

Agenda

0 Ouverture — 15 min

1 Comprendre la machine — 40 min

2 Cadrer avant de coder — 45 min, *puis pause 10 min*

3 Produire et vérifier — 45 min

4 Étendre l'agent — 35 min

5 Garder la main — 20 min, *puis clôture 15 min*

© ZENIKA 2026 | All rights reserved

Ouverture · 15 min

Vibe coding ou AI-driven dev ?

Ce qu'on vise : un agent qui produit vite, un dev qui garde la main.



© ZENIKA 2026 | All rights reserved

Vibe coding vs AI-driven dev

	Vibe coding	AI-driven dev
Logique	explorer vite	produire de façon fiable
Marche bien pour	spike, prototype, script jetable	code livré, relu, maintenu
Exemples	tester une lib, maquetter un écran	une feature avec ses tests, un refactor
Risque accepté	flou, variance, déchet	peu de flou, traçabilité
Réflexe	on tente, on observe, on jette	on cadre, on contrôle, on capitalise

i Les deux sont utiles : **viber pour explorer, piloter pour livrer.**

© ZENIKA 2026 | All rights reserved

Une brève histoire

2022 — **GitHub Copilot** : l'autocomplétion dans l'IDE.

2023 — **ChatGPT** se popularise le shadow AI pour coder.

Fin 2024 — **Cursor** rebat les cartes, les outils deviennent agnostiques du modèle. **MCP** est publié.

Fév. 2025 — **Claude Code** : l'agent en CLI, avec un mode plan.

Fin 2025 — Copilot, Cursor, Windsurf : tous un **mode plan** et une **CLI**.

Nov. 2025 — **Gemini 3 Pro** : Google revient parmi les meilleurs modèles.

2026 — Agents en CLI et dans le cloud **AGENTS.md**, **SKILL.md** et **MCP** partagés par les harness.

> Le paysage bouge vite : on apprend des **techniques**, pas un outil figé.

© ZENIKA 2026 | All rights reserved

Sortie subie ou sortie pilotée ?

Une scène familière

Ajoute la pagination sur l'endpoint /bookings.

[pagination offset, nouveau helper maison]

Non, on utilise des curseurs.

[curseurs, mais un format de page inventé]

Tu as oublié les tests…

[tests qui mockent tout]

Recommence, plus simple.

Ce qu'il fallait donner dès le départ

- « ajoute la pagination » → un **objectif** et un **critère de fin**
- « on utilise des curseurs » → les **conventions** du projet
- « le format de page commun » → les **invariants**
- « les tests » → les **commandes de vérification**

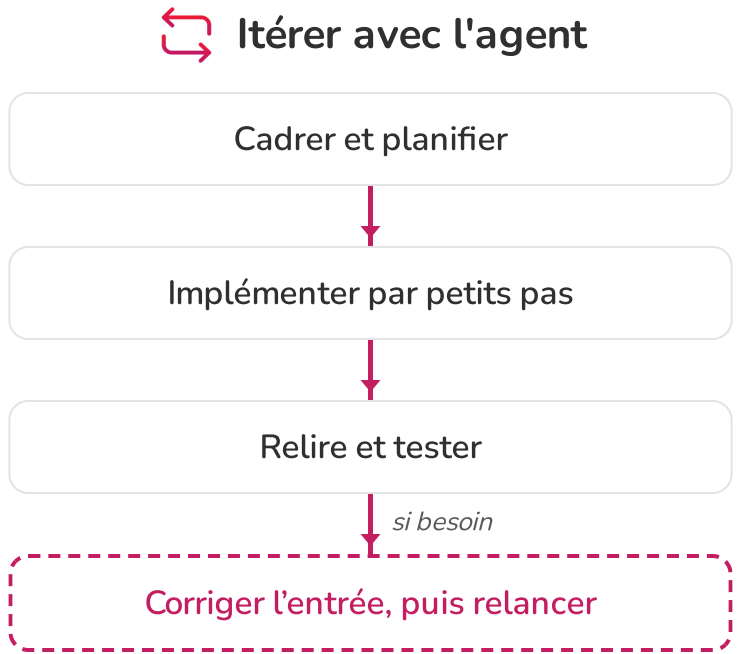
i 4 allers-retours : une **sortie subie**. Pas de prompt magique : on construit l'entrée pour une **sortie pilotée**.

© ZENIKA 2026 | All rights reserved

Le développeur comme orchestrateur

Avec un agent de code, le rôle du développeur se déplace :

- d'écrire chaque ligne → cadrer, diriger, relire,
- de mémoriser la syntaxe → exprimer clairement son intention,
- de chercher la réponse → vérifier celle qu'on reçoit.



© ZENIKA 2026 | All rights reserved

Quiz de positionnement

QUIZ 3 min

	Vrai ou faux ?	
1	Le modèle lance lui-même `npm test` dans le terminal.	Faux
2	La même demande produit toujours le même code.	Faux
3	Plus on donne de contexte à l'agent, mieux il travaille.	Faux
4	Le fichier d'instructions projet est relu à chaque session.	Vrai
5	Si les tests passent, le code de l'agent est bon.	Faux

© ZENIKA 2026 | All rights reserved

Module 1 · 40 min

Comprendre la machine

Ce que fait vraiment un LLM, et ce qu’est un agent de code.



© ZENIKA 2026 | All rights reserved

Six notions, une phrase chacune

LLM · LARGE LANGUAGE MODEL

Le moteur

Un réseau de neurones qui prédit le token suivant, à partir de tout ce qu'on lui envoie.

GPT-5 · Claude · Gemini · Mistral

TOKEN

L'unité de compte

Un morceau de mot ou de code. Le modèle lit, écrit, limite et facture en tokens.

CONTEXT WINDOW

Tout ce que le modèle voit

Le texte envoyé à chaque appel : consignes, fichiers, historique. Ce qui n'y est pas n'existe pas.

HARNESS

Tout sauf le modèle

Le logiciel qui entoure le modèle : boucle, tools, contexte, permissions.

Claude Code · Copilot · Codex · Cursor

TOOLS

Les mains

Lire un fichier, lancer une commande, chercher : le modèle les demande, le harness les exécute.

AGENT

Le modèle qui agit

Un modèle placé dans une boucle avec des tools et un objectif : il choisit lui-même la prochaine action.

© ZENIKA 2026 | All rights reserved

Le texte découpé en tokens

Le modèle ne lit pas des mots, mais des **tokens** :

Français	`Ajoute la pagination sur l'endpoint /bookings.`	6 mots → **12 tokens**
Anglais	`Add pagination to the /bookings endpoint.`	6 mots → **9 tokens**
Mot rare	`anticonstitutionnellement`	1 mot → **5 tokens**
Code	`return res.status(404).json({ error: "Not found" });`	**15 tokens**

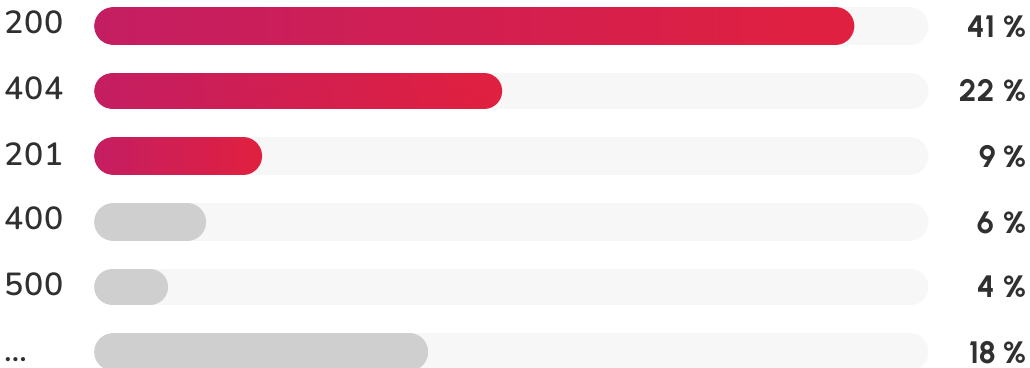
- Les mots **courants** tiennent en un token, les mots **rares** et les identifiants sont découpés.
- Le **français** consomme plus de tokens que l'anglais pour dire la même chose.

> *i* Plus de tokens = **plus cher**, **plus lent**, et **plus de place** prise dans la context window.

© ZENIKA 2026 | All rights reserved

Choisir le token suivant : plausible ≠ vrai

Après `return res.status(…)`



Probabilités illustratives du token suivant. Le modèle tire au sein des candidats les plus probables.

UNE PART DE HASARD

Il tire parmi les candidats **les plus probables** : relancer donne une **autre réponse**.

CHAQUE MOT PÈSE

`curseur`, `OpenAPI`, `avec tests` : quelques mots de la demande déplacent toute la suite.

Un LLM produit du plausible, pas du vrai. Sans la bonne information dans le contexte, l'hallucination est le fonctionnement normal.

© ZENIKA 2026 | All rights reserved

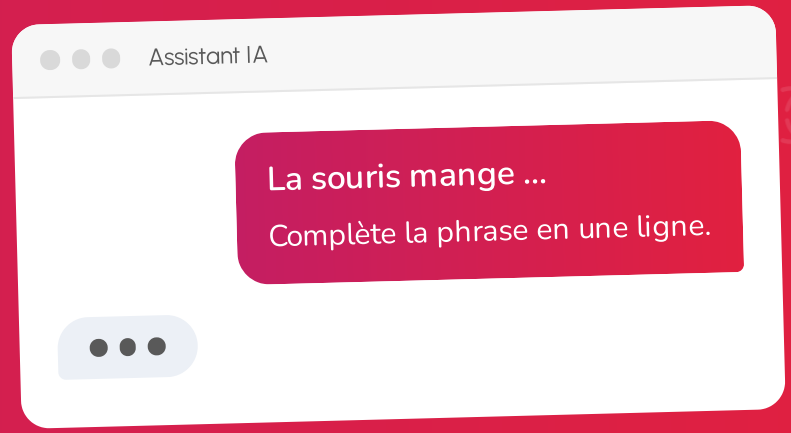
DÉMO 3 min ↗

▶ DÉMO

La souris mange…

Un prompt, trois essais, un mot de plus.

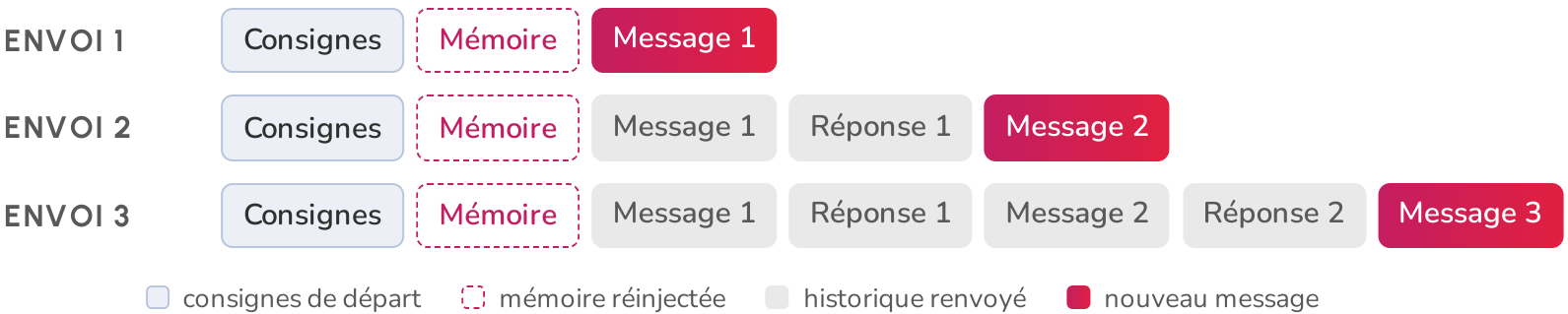
Le modèle choisit la suite la plus probable, pas la plus vraie.



© ZENIKA 2026 | All rights reserved

Le modèle n’a pas de mémoire

À chaque appel, le harness renvoie **toute la session** au modèle.



- Une erreur du tour 2 est **relue à chaque tour** suivant.
- La « mémoire » du harness (fichiers d’instructions, notes gardées) est du texte **relu à chaque session** : une info périmée continue de peser.

> *i* Une **nouvelle session** repart de zéro : c’est un outil, pas un échec.

© ZENIKA 2026 | All rights reserved

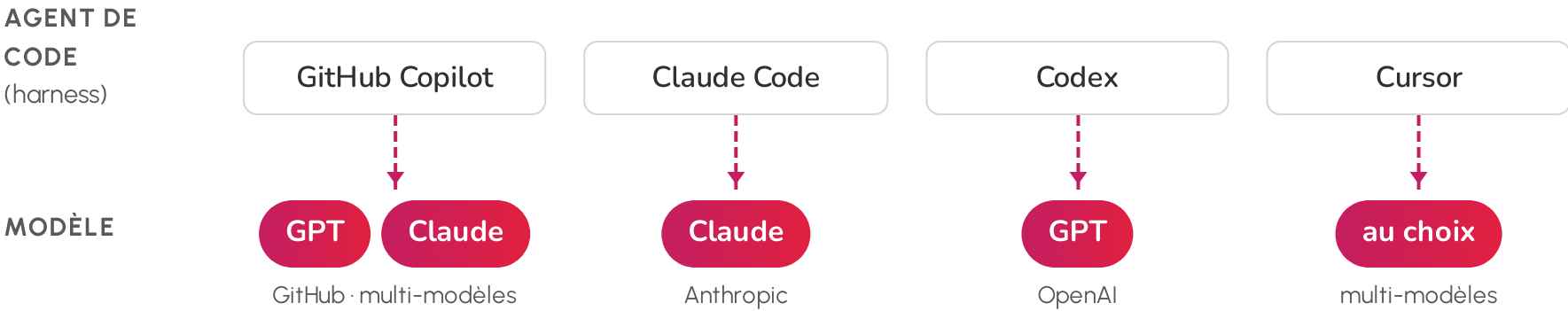
Chatbot, workflow ou agent ?

	Chatbot	Workflow LLM	Agent
Qui décide de l'étape suivante ?	l'humain	le code (chemin écrit à l'avance)	le modèle
Tools	aucun ou peu	appelés à des étapes fixes	choisis à chaque tour
Prévisibilité / coût	élevée / faible	élevée / faible	faible / variable
Exemple	« Explique-moi cette regex »	résumer chaque PR à l'ouverture	« Ajoute la pagination à /bookings, avec ses tests »

i Commencer par **la solution la plus simple** qui marche. Un workflow prévisible bat souvent un agent autonome.

© ZENIKA 2026 | All rights reserved

Modèle ≠ assistant (le harness)



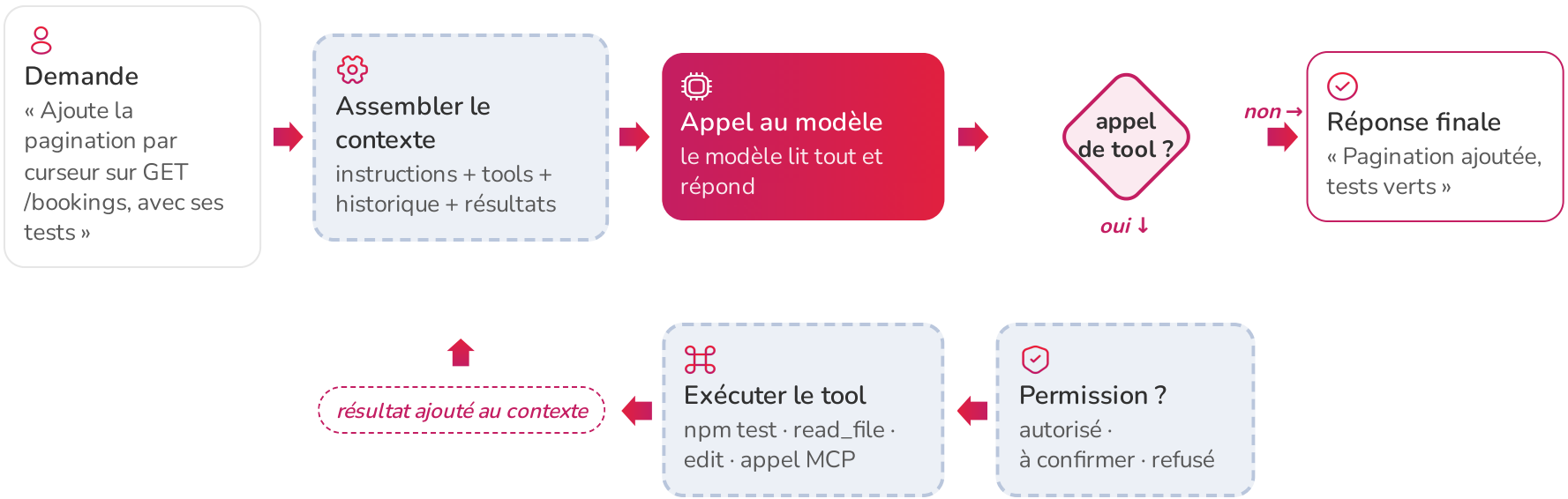
- Le **modèle** est le moteur ; le **harness** lui donne les moyens d'agir : boucle de tools, contexte, permissions, traces.
- **Même modèle, deux harness** : des résultats très différents. Le harness pèse autant que le choix du modèle.

i Les noms changent tous les trimestres. Les notions restent.

© ZENIKA 2026 | All rights reserved

La boucle agentique

Agent = modèle + tools + boucle + objectif. Le modèle n'exécute rien : il écrit une **demande d'action**, le harness l'exécute.



Un tour = un appel au modèle. Une tâche simple : 5 à 20 tours. Une grosse tâche : des centaines.

© ZENIKA 2026 | All rights reserved

Quiz : qui est qui ?

QUIZ 3 min

Claude Sonnet LLM	Claude Code harness	Copilot, Tab pour accepter complétion	Copilot en mode agent agent
read_file tool	MCP protocole	AGENTS.md instructions projet	SKILL.md skill
« Résume chaque PR » workflow	Codex CLI harness	GPT-5 LLM	Copilot coding agent agent cloud

© ZENIKA 2026 | All rights reserved

Une trace d’agent, commentée

TIPS

Tour	Le modèle demande	Le harness renvoie	Contexte
1	`grep: "bookings"`	6 fichiers	12 k
2	`read_file: bookings.controller.ts`	20 lignes	14 k
3	`read_file: shared/pagination/cursor.ts`	100 lignes	15 k
4	`edit: bookings.controller.ts` + test	diff appliqué, après accord	17 k
5	`bash: npm test -- bookings`	échec : `nextCursor is undefined`	19 k
6	`edit: bookings.controller.ts`	diff appliqué	20 k
7	`bash: npm test -- bookings`	succès : `12 passed`	21 k

TIP Le code généré est une proposition plausible, pas une vérité.

© ZENIKA 2026 | All rights reserved

Module 2 · 45 min

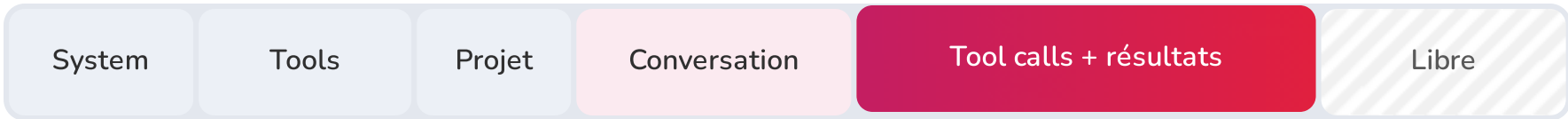
Cadrer avant de coder

Ce qu’il faut donner à l’agent avant qu’il touche au code.



© ZENIKA 2026 | All rights reserved

La context window d’un agent de code



CONTEXT WINDOW D’UN AGENT DE CODE — TOUT CE QUE LE MODÈLE REÇOIT À CHAQUE APPEL

Tool calls + résultats

Fichiers lus, sorties de commandes, logs d’erreur, résultats de tests : le bloc qui grossit le plus vite.

C’est **tout** ce que le modèle voit quand il répond : consignes du harness, tools, instructions projet, conversation, fichiers lus, sorties de commandes.

i Chaque tool exposé, chaque fichier lu, chaque sortie de commande **consomme du budget.**

© ZENIKA 2026 | All rights reserved

Vue d'ensemble : l'agent de code



© ZENIKA 2026 | All rights reserved

Instructions projet (`AGENTS.md`)

Un fichier chargé à chaque session

Versionné à la racine : conventions, commandes, pièges. **Court**, avec des règles **vérifiables**.

Vague	Concret
« Formate proprement »	*« Indentation 2 espaces »*
« Teste tes changements »	*« npm test avant chaque commit »*
« Range le code »	*« Handlers dans src/api/handlers/ »*

Le générer, puis le relire

L'agent analyse le dépôt et propose

↓

Relire : concepts métier, invariants

↓

Élaguer ce qui se déduit du code

↓ *votre expertise*

Ajouter les pièges connus

i La génération donne un **point de départ**. Votre expertise **comble les trous**, surtout les pièges.

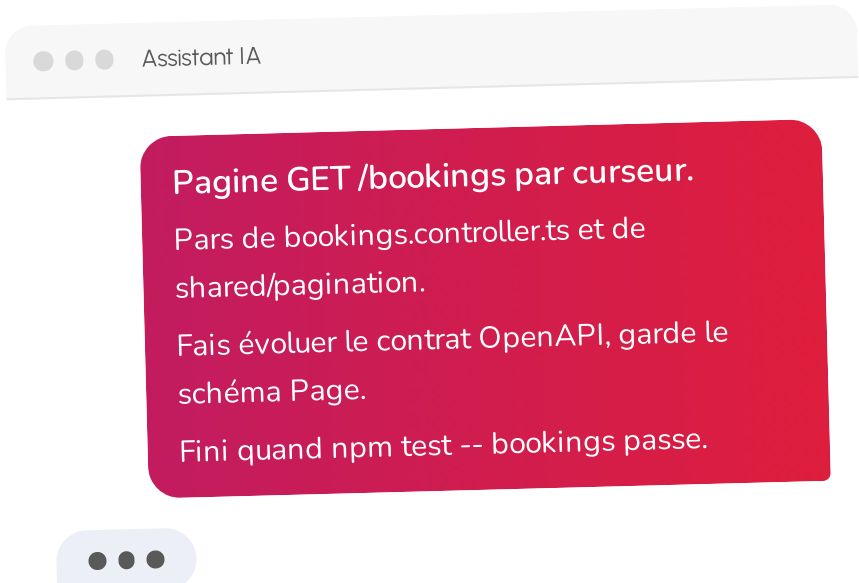
© ZENIKA 2026 | All rights reserved

Une demande initiale bien cadrée

Le principe

- Le premier message de la session : il oriente **toute la suite**.
- **Court et relu** : objectif, fichiers, invariants, vérification.
- Les noms sont des **ancres** : un chemin, une commande, un invariant orientent l'attention.

Exemple



i Tâche longue ? On l'écrit dans un **fichier**, rechargé à chaque session.

© ZENIKA 2026 | All rights reserved

Planifier avant de coder

L'erreur n° 1 : se jeter directement dans la génération. Le mode plan des harness explore et propose, sans modifier de fichier.

1



Besoin flou

Le ticket tel qu'on le formule en une phrase.

EXEMPLE

« Pagine /bookings. »

2



Plan relu

L'agent propose la demande cadrée et les étapes ; vous les corrigez et les validez.

EXEMPLE

4 étapes vérifiables, réponse { items, nextCursor }, contrat OpenAPI mis à jour, schéma Page inchangé.

3



Code

Généré à partir du plan validé, pas du besoin flou.

EXEMPLE

Controller + tests, une étape à la fois.

TIP Fais valider la demande et le plan avant de laisser l'agent écrire du code : la sortie d'une étape devient le contexte de la suivante.

© ZENIKA 2026 | All rights reserved

Modes de pilotage : monter l'autonomie

Tous les harness proposent les **mêmes niveaux d'autonomie**, sous des noms différents : mode plan, demander, accepter les éditions, tout autoriser…

Niveau d'autonomie	L'agent…	Pour…
Observer sans agir	lit, cherche, propose un plan	explorer un code inconnu
Demander à chaque action	attend ton accord à chaque édition	apprendre à lire ce qu'il propose
Agir dans le périmètre	modifie les fichiers, demande pour le reste	dérouler un plan validé
Aller au bout seul	enchaîne sans rien demander	un environnement isolé et jetable

i On monte d'un niveau quand on fait confiance au **périmètre**, avec un **filet** : tests, commits fréquents, diff relu.

© ZENIKA 2026 | All rights reserved

Exercice 1 — Du ticket au plan

OPTIONNEL 15 min ↗

Concept : cadrer avant de coder.

Demande à ton agent…

1. **sans rien modifier**, de proposer la **demande initiale** et un **plan** pour `TICKET.md` ;
2. … puis **corrige-les**.

Réussi quand…

- **aucun fichier** n'a été modifié ;
- au moins **une correction** vient de toi.

© ZENIKA 2026 | All rights reserved

Pause · 10 min

On souffle



© ZENIKA 2026 | All rights reserved

Module 3 · 45 min

Produire et vérifier

Avancer par petits pas sans perdre la maîtrise du code.



© ZENIKA 2026 | All rights reserved

Feature par feature : la boucle

Préparer

1. Lire le ticket
2. Faire comprendre le code
3. Demande et plan validés

Livrer, puis feature suivante

4. Implémenter une étape
5. Tester et relire

vert et relu (sinon retour à 4)

6. Commit : une étape = un commit
7. Étape suivante du plan
8. Mettre à jour la doc projet

© ZENIKA 2026 | All rights reserved

Rewind plutôt qu'empiler



i Traitez une mauvaise sortie comme un **message du compilateur** : on corrige l'entrée, on relance.

© ZENIKA 2026 | All rights reserved

Relire le code généré

Le code de l'agent se relit comme celui d'un **contributeur inconnu**.

CORRECTION

Résout-il vraiment le problème du ticket ?

CAS LIMITES

Liste vide, `null`, dernière page, grandes valeurs.

SÉCURITÉ

Entrées validées, autorisations correctes.

PERFORMANCE

Requêtes N+1, boucles, fuites mémoire.

LISIBILITÉ

Compréhensible dans 6 mois, sans sur-ingénierie.

DÉPENDANCES

Les imports et les API **existent-ils vraiment** ?

> *i* Le vrai danger : l'**incompétence cachée**, du code qui a l'air bon. L'agent peut se relire, mais c'est **vous** qui décidez.

© ZENIKA 2026 | All rights reserved

Deux natures de tests, deux publics

	Tests de construction	Tests fonctionnels
À qui ils servent	à l'agent, pour se contrôler pendant qu'il code	à l'équipe, pour savoir ce que fait le produit
Ce qu'ils testent	l'implémentation, unité par unité	le comportement observable de l'API
Durée de vie	jetables	durables : ils *sont* la spec exécutable



En fin de feature, là où les deux se recouvrent, **le test fonctionnel gagne** : on supprime le doublon.

© ZENIKA 2026 | All rights reserved

Brownfield : comprendre et figer avant de modifier

Pas tout le code d'un coup

- 500 000 lignes ≈ des millions de tokens : ça ne rentre pas.
- L'agent navigue : recherche, lecture ciblée.
- On attaque par morceaux : un module, un endpoint.

Figer le comportement actuel

```
Avant de toucher à BookingsController,
écris des tests qui capturent le
comportement ACTUEL de GET /bookings,
même s'il est faux.
```

Ces tests de caractérisation détectent le changement, pas la correction.

i **Faire comprendre avant de faire modifier** : une explication relue vaut mieux qu'un diff incompris.

© ZENIKA 2026 | All rights reserved

Revue IA : donner le contexte du diff

Donner à l'agent relecteur	Pourquoi
Le diff complet, avec du contexte autour	voir la fonction entière, pas trois lignes
Le ticket, la demande initiale, le plan	juger le code sur l'intention
Les conventions (instructions projet)	relever les écarts propres à l'équipe
La grille « Relire le code généré »	chercher les défauts, pas complimenter

TIP Ce que tu ne relis pas, tu le livres quand même. La revue IA aide, un humain valide avant le merge.

© ZENIKA 2026 | All rights reserved

Exercice 2 — Petits pas, tests, revue

OPTIONNEL 20 min ↗

Concept : produire et vérifier.

Demande à ton agent…

1. d'implémenter **une étape** du plan, **avec ses tests** ;
2. … puis **relis le diff** avec la grille, et fais-le relire par l'agent.

Pas de plan ? Pars de `docs/plan-de-secours.md`.

Réussi quand…

- les **tests passent** ;
- tu as repéré **un défaut** de l'agent.

© ZENIKA 2026 | All rights reserved

Module 4 · 35 min

Étendre l’agent

Brancher l’agent sur le reste du SI, et capitaliser ce qui marche.



© ZENIKA 2026 | All rights reserved

Apporter le contexte de l'entreprise

Deux façons de donner à l'agent ce que **le dépôt ne contient pas** :



RAG : vos documents

Apporte de la connaissance : ADR, docs d'architecture, wiki, conventions d'API.

> *« Quelle est notre convention de pagination ? »*

VIGILANCE

Des documents à jour, sinon des réponses périmées… mais sourcées.



Tools / MCP : vos applications

MCP, la prise standard des tools (l'« USB-C » des agents) : tickets, base, CI, navigateur.

> *« Lis le ticket BOOK-142, puis lance les tests e2e dans le navigateur. »*

VIGILANCE

Des droits limités : l'agent agit avec vos accès.

i Dans les deux cas, l'agent ne **sait** rien de plus : on lui **donne** le bon contexte, au bon moment.

© ZENIKA 2026 | All rights reserved

MCP : ce que vous exposez en branchant un serveur

Brancher un serveur, c'est laisser le modèle **décider seul** quelles données partent chez un tiers.

QUI L'EXÉCUTE ?

Local ou distant

Le local reste sur la machine. Le distant reçoit ce que le modèle lui envoie.

QUI L'AUTHENTIFIE ?

Quel jeton, quels droits

Lecture, écriture, suppression : jamais plus large que la tâche.

QUI LE MAINTIENT ?

Officiel ou fork

Un serveur non maintenu est un risque ouvert.

ET CE QU'IL COÛTE

Chaque serveur ajoute ses tool definitions à **chaque** appel. N'activez que ceux que vous utilisez **vraiment** pour ce projet.

© ZENIKA 2026 | All rights reserved

Une bonne consigne devient une skill

Un savoir-faire emballé

- Un **dossier** : un `SKILL.md`, plus des scripts, références et gabarits.
- Chargée **à la demande**, quand la tâche correspond : la **progressive disclosure**.
- La **description** dit quoi et quand : c'est elle qui déclenche.
- Testée en **session vierge** : une autre session la suit sans coaching.

Exemple : la skill « tests-api »

```
tests-api/SKILL.md

name: tests-api
description: Écrire ou compléter les tests
  d'un endpoint. À utiliser dès qu'on
  ajoute ou modifie une route.

- Un test par cas limite : liste vide,
  dernière page, curseur invalide.
- Données via tests/factories.
- Base de test réelle, pas de mock.
```

TIP Une consigne que tu répètes trois fois mérite une skill. Elle ne pèse que quand elle sert.

© ZENIKA 2026 | All rights reserved

Quiz : où ranger le contexte durable ?

QUIZ 3 min

« Les tests se lancent avec npm test, jamais sur la base de prod »

instructions projet

« Pour ce ticket : garder le schéma Page de /rooms »

demande initiale

« Comment écrire un test d'API selon nos conventions »

skill

« Étapes 1 à 3 faites, l'étape 4 bloque sur le curseur »

note de reprise

« TypeScript strict, handlers dans src/api/handlers »

instructions projet

« La procédure de migration de schéma, avec son script »

skill

i Plus c'est **chargé souvent**, plus c'est **court** : à chaque session, pour une tâche, à la demande.

© ZENIKA 2026 | All rights reserved

Exercice 3 — Ma première skill

OPTIONNEL 10 min ↗

Concept : capitaliser.

Demande à ton agent…

1. d'écrire en `SKILL.md` une consigne que tu répètes ;
2. … puis teste-la en **session vierge**.

Pas d'idée ? « Nos tests utilisent `tests/factories`, sans mock. »

Réussi quand…

- la skill **se déclenche** en session vierge.

© ZENIKA 2026 | All rights reserved

Module 5 · 20 min

Garder la main

Ce qui peut mal tourner : contexte, sécurité, coût.



© ZENIKA 2026 | All rights reserved

Plus n’est pas mieux

QUIZ 2 min

SITUATION 1

« Explique-moi cette regex », dans un chat neuf.

simple

Attention concentrée.

SITUATION 2

L'agent a lu 14 fichiers et lancé 3 fois les tests.

chargée

Les tools dominent la demande.

SITUATION 3

Tour 40, objectif changé deux fois, log de 3 000 lignes.

polluée

Crédible mais faux.

SITUATION 4

Session neuve, demande cadrée, plan validé.

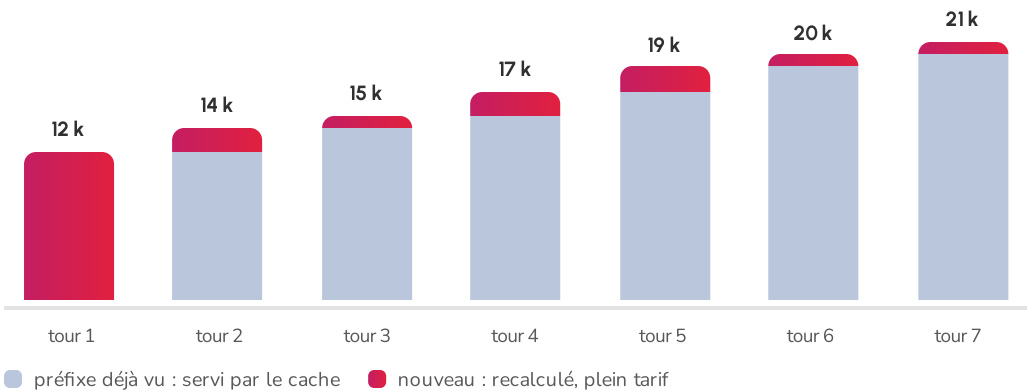
préparée

Moins d'exploration à l'aveugle.

La bonne question : **tout ce qui reste ici aide-t-il encore la tâche du jour ?** Sinon, une **session neuve**.

© ZENIKA 2026 | All rights reserved

Coûts : le token se paie



118 k tokens d'entrée envoyés au total, dont 82 % lus en cache

5 réflexes

1. 1 tâche = 1 session : moins d'historique renvoyé.
2. Être précis : une demande cadrée, moins d'exploration.
3. Couper les MCP inutiles : chacun pèse dans le contexte.
4. Garder le même modèle : sinon le cache est perdu.
5. Demander du concis : moins de tokens en sortie.

i Le token se paie, en **entrée** comme en **sortie** : le préfixe déjà vu est **renvoyé à chaque tour**.

© ZENIKA 2026 | All rights reserved

5 risques, 5 garde-fous



Hallucination

Une API ou un paquet inventés, mais plausibles.

> *Un paquet inventé par l'IA, publié ensuite par un tiers.*

GARDE-FOU

Tests, typage, relecture



Prompt injection

Un contenu piégé détourne l'agent.

> *Une issue publique qui fait fuiter un dépôt privé.*

GARDE-FOU

Accès de l'agent limités



Fuite de données

Code, secrets ou données sortent de l'entreprise.

> *Du code propriétaire collé dans un chatbot.*

GARDE-FOU

Outils validés, pas de secret



Action destructrice

L'agent supprime ou écrase ce qu'il ne fallait pas.

> *Une base de production effacée par un agent.*

GARDE-FOU

Sandbox, humain qui valide



Coûts

Une consommation qui s'emballe.

> *Des clés volées, des factures à 5 chiffres.*

GARDE-FOU

Plafonds, suivi

i **Trio létal** : données privées + contenu non fiable + sortie vers l'extérieur = exfiltration possible. **Casser** au moins une branche.

© ZENIKA 2026 | All rights reserved

2 règles d’or

1



Code et données sensibles : outils validés

Rien de confidentiel hors des harness et modèles validés par l’entreprise : code client, données de prod, secrets (.env, jetons).

LE TEST

L’enverriez-vous par email à un inconnu ?

2



Un humain valide avant le merge

Obligatoire pour ce qui est irréversible : merge, déploiement, migration, push forcé. Le code généré est le vôtre.

LE TEST

Qui signe si c’est faux ?

i Ces deux règles couvrent l’essentiel des risques : la **fuite** d’un côté, l’**erreur** de l’autre. C’est le **human-in-the-loop**.

© ZENIKA 2026 | All rights reserved

Quiz : safe ou pas safe ?

QUIZ 3 min

Coller le .env dans le chat pour que l'agent comprenne la config.

pas safe

L'agent explore le dépôt en lecture seule pour préparer la demande.

safe

L'agent ajoute un paquet que personne dans l'équipe ne connaît.

pas safe

Un agent lit les issues publiques, accède au dépôt privé et peut commenter.

pas safe

L'agent va au bout seul, dans un conteneur jetable sans secret.

safe

Relire le code d'un client avec son abonnement perso à un chatbot.

pas safe

TIP Une tâche, une session. Jamais de secret dans le contexte.

© ZENIKA 2026 | All rights reserved

Clôture · 15 min

Ce qu’on garde

Les réflexes à garder lundi matin.



© ZENIKA 2026 | All rights reserved

Un prototype n’est pas un produit

« Il marche, ce que l’agent a généré en deux heures… **on le merge ?** »

Prototype — vibe coding	Produit — AI-driven development
DONNÉES *Fictives, sans droits d’accès*	**DONNÉES** *Réelles : qui voit quoi, pas de fuite*
TESTS *« Ça marche sur ma machine »*	**FIABILITÉ** *Tests relus, cas limites, retour arrière possible*
ERREURS *On relance*	**SÉCURITÉ ET MONITORING** *Accès contrôlés, erreurs détectées et suivies*
ÉVOLUTION *On régénère*	**MAINTENANCE** *Une équipe le comprend et le fait évoluer*

i Le vibe coding produit un **prototype** à jeter ou à reprendre. Ce qu’on livre se **cadre, se teste et se relit**.

© ZENIKA 2026 | All rights reserved

6 réflexes pour lundi matin

TIPS

PLAUSIBLE ≠ VRAI

Vérifier

Le modèle prédit le probable. Tests, typage et revue restent les garde-fous.

CADRER

Demande et plan d'abord

Faire valider la demande initiale et le plan avant de laisser l'agent écrire.

AUTORÉGRESSION

Rewind

Une sortie fausse : on corrige l'entrée et on relance, au lieu d'empiler.

SANS MÉMOIRE

Une tâche, une session

Ce qui n'est pas dans la context window n'existe pas, ce qui y traîne pollue.

RESPONSABILITÉ

Relire le diff

Petits pas, un commit par étape, une relecture humaine avant le merge.

CAPITALISER

Extraire une skill

La consigne qu'on répète devient une skill, testée en session vierge.

© ZENIKA 2026 | All rights reserved

Quiz de sortie

QUIZ 5 min

Un token ?

L'unité que le modèle lit, écrit et facture.

La context window ?

Tout ce que le modèle reçoit à un appel.

Le harness ?

Tout sauf le modèle : boucle, tools, permissions.

Agent ou prompt ?

Un prompt demande une réponse, un agent poursuit un objectif en boucle.

MCP ?

Un protocole standard pour brancher des tools.

Une skill ?

Un savoir-faire chargé quand la tâche correspond.

AGENTS.md ou demande initiale ?

Chaque session, court
La tâche du jour

Qui répond du code généré ?

Le dev qui l'a relu et mergé.

© ZENIKA 2026 | All rights reserved

Questions

Quel réflexe appliquez-vous à votre prochain ticket ?

© ZENIKA 2026 | All rights reserved